CS50, Vid2

## Shorts 1: Functions

- Some piece of code that takes an input and produces an output

ex. the print function

```
print("yo.")
```

← this function named "print" is called.
print is a built-in function within python.

You can identify if something is a function by looking for identifiers

```
print()
```

← these parentheses are identifiers that "print" is a function that will take in some input.

Some functions might not even take any input.

ex. `print()` ← this means this function does not take an input, or I have not given an input to this function.

To make my own functions, I use "def" to define my function

ex.
```
def main():
    print("yo.")
    print("what's up")
main()
```

here, the function "main" does NOT take in a function, but it DOES run the two print functions.

When asking who created the "print" function, or who defined the print function, the python developers defined what happens in the print function somewhere within the python library.

## Shorts 2: Variables

- Containers for some values that can change over time
- we assign values to variables using the "="

ex.
```
def get_guess():
    guess = 10
    return guess
print(get_guess())
```

} the function "get_guess" simply holds a value by assigning a variable "guess" with a value "10"

← the function "get_guess" will return the value to a different function

get_guess will return the value to a print function

Comparing values

ex.
```
def get_guess():
    guess = input("guess here: ")
    return int(guess)

def main():
    guess = get_guess()
    if guess == 50:
        print("correct")
    else:
        print("incorrect")

main()
```

the variable "guess" in the function "get_guess" is different from the variable "guess" in the function "main" (different scopes)

if else statement:
if guess is equal to 50, the function will print "correct."
else (if guess is not equal to 50 and is something else completely) the function will print "incorrect"

## Shorts 3: Return values

Functions can produce two different types of outputs:
return values and side effects

ex.
```
def greet(input):
    if "hello" in input:
        print("hi")
    else:
        print("what?")
greet("hello, comp")
```

hi ← this is a side effect: some stimuli (visual here) that I can receive physically. This is not a return value

Return value
- Some value a function can pass to a program to use later on in the code
to return some value, we use the function "return"

ex.
```
def greet(input):
    if "hello" in input:
        return "hi"
    else:
        return "what?"
greet("hello, comp")
```

[there is nothing here, but the function "greet" is returning some value. We are not yet using the values.]

these are return values

To capture and use the values, we need to store it into a variable

ex.
```
def greet(input):
    if "hello" in input:
        return "hi"
    else:
        return "what?"
greeting = greet("hello, comp")
print(greeting)
```

hi ← this is a side effect

these are return values

function stores return values into the variable "greeting"

print function allows the value to be displayed by presenting the value stored in the variable "greeting"

return values allow the user to modify the code and use the output of some code later on.

Side effects simply produces some change immediately, while return values are usable throughout a program

## Shorts 4: Side effects

Side effects are anything that gets changed while a function is running.
An example of a side effect includes printing to the terminal.
Essentially, anything we change in the program and see the product of that change is a side effect.
Another example of a side effect is one that changes some global variable.

ex. emotion = "UwU"

```
def pathosmeter():
    global emotion
    Say("hello?")
    emotion = ";D"
    Say("yo!")
def Say(phrase):
    print(phrase + " " + emotion)
```

```
hello? UwU
yo! ;D
```

the keyword "global" allows the user to specify a global variable that is normally local inside a function.

The global variable is now both accessible and modifiable.

accessible, meaning we can access the variable "emotion" in the function "Say"

modifiable, meaning we can now modify and update the variable "emotion" in the function "pathosmeter" from a local variable to a global variable.

"global emotion" modifies the actual value of that variable as a side effect

## Shorts 5: String Methods

methods refer to functions that belong to some kind of object

Some methods for Strings:

.capitalize()
- uppercases the first character in a given String

.title()
- title cases the words in a given string

.strip()
- removes spaces in a string on both left and right sides
- use .lstrip() to remove spaces on the left, .rstrip() to remove spaces on the right

.append()
- Strings are moved to a new variable

.join()
- Specifies a String you want to use to join some String elements with

String methods can be chained together